

Estándares de Arquitectura y Enrutamiento (Guía para el Equipo de Desarrollo)

Este documento establece las reglas fundamentales de programación y la estructura de rutas (URLs) para todo el equipo de SGDM. **Es obligatorio seguir estos lineamientos** para evitar conflictos (Merge Hell) y garantizar que todos los módulos se integren perfectamente en el Front Controller ([index.php](#)).

REGLAS DE ORO DEL EQUIPO

1. Nomenclatura de Controladores (Ruteo Estricto)

Nuestro sistema utiliza un [index.php](#) centralizado que enruta las peticiones mediante las variables `?c=` (Controlador) y `a=` (Acción).

- **Regla:** Todos los archivos de controladores deben guardarse en la carpeta [controladores/](#).
- **Regla:** El nombre del archivo y la clase debe terminar obligatoriamente con la palabra [Controlador](#) (Ejemplo: [TorneoControlador.php](#)).
- **Regla:** La línea 2 de **todo controlador** debe ser: `namespace App\Controladores;`

2. Base de Datos Única (Modelos)

- Las tablas SQL son compartidas. **No creen modelos duplicados** para acceder a la misma tabla.
- Si necesitas operar sobre los usuarios, abre [modelos/Usuario.php](#) y añade tu método allí (no crees un "JugadorModelo.php").
- **Regla:** Todo modelo nuevo debe ir en [modelos/](#) y tener `namespace App\Modelos;` en la línea 2.
- **Regla:** Todo modelo debe implementar la conexión a la base de datos de manera segura mediante sentencias preparadas (PDO).

3. Libertad Frontend (Vistas)

- Tienen libertad para crear carpetas, archivos HTML, e inyectar clases de CSS a discreción dentro de sus directorios de Vistas asignados ([vistas/jugador/](#), [vistas/organizador/](#), [vistas/publico/](#)).
- **Regla:** Nunca sobrescriban los archivos de la carpeta [vistas/admin/](#).

4. Flujo de Trabajo en Git

- Si necesitan modificar archivos "core" del sistema (como [publico/index.php](#), [ayudantes/Sesion.php](#) o [estilos.css](#)), **comuníquenlo al equipo primero** para evitar conflictos de fusión irresolubles.

MAPA DE RUTAS APROBADAS (UML)

A continuación se define la estructura de URL (Controladores y Acciones) que cada desarrollador debe programar. Cuando programen un botón en HTML, el enlace `<a href>` debe apuntar obligatoriamente a estas rutas.

● MÓDULO PÚBLICO (Sin Login)

Estas rutas procesan la navegación de un visitante casual.

- **Página de Inicio (Landing Page):** `?c=inicio&a=index`
- **Registro de Usuario:** `?c=auth&a=mostrarRegistro`
- **Procesar Registro:** `?c=auth&a=procesarRegistro`
- **Catálogo de Torneos Público:** `?c=torneo&a=catalogoPublico`

● MÓDULO JUGADOR (Rol 3)

Estas rutas manejan la experiencia del competidor. *Todas deben estar protegidas verificando el rol.*

- **Inicio / Dashboard Jugador:** `?c=jugador&a=dashboard`
- **Editar Perfil Propio:** `?c=perfil&a=editar`
- **Ver Mis Equipos:** `?c=equipo&a=index`
- **Crear Nuevo Equipo:** `?c=equipo&a=crear`
- **Buscar Torneos para Inscribirse:** `?c=torneo&a=explorar`
- **Inscribirse a Torneo:** `?c=inscripcion&a=nueva`

● MÓDULO ORGANIZADOR (Rol 2)

Estas rutas permiten gestionar competencias de forma independiente. *Todas deben estar protegidas verificando el rol.*

- **Inicio / Dashboard Organizador:** `?c=organizador&a=dashboard`
- **Ver Mis Torneos:** `?c=torneo&a=misTorneos`
- **Crear un Torneo:** `?c=torneo&a=crear`
- **Bandeja de Solicitudes (Inscripciones pendientes):** `?c=inscripcion&a=pendientes`
- **Aprobar / Rechazar Inscripción:** `?c=inscripcion&a=gestionarEstado`
- **Panel de Fixture / Llaves:** `?c=partida&a=fixture`
- **Cargar Resultado de Partida:** `?c=partida&a=cargarResultado`

● MÓDULO ADMINISTRADOR (Rol 1 - Core)

(Ya desarrollado o en desarrollo por el líder técnico)

- **Dashboard Admin:** `?c=admin&a=dashboard`
- **Gestión de Usuarios:** `?c=usuario&a=index`
- **Catálogo de Juegos:** `?c=juego&a=index`

🔒 BUENAS PRÁCTICAS Y SEGURIDAD (Para Desarrolladores Junior)

Ya que todos estamos aprendiendo y construyendo esto juntos, estas son las 5 reglas para evitar que el sistema sea hackeado o se vuelva imposible de leer:

1. Nunca confíen en el usuario (Prevención de Inyección SQL)

Jamás metan una variable `$_POST` o `$_GET` directamente adentro de un texto SQL.

- **✗ Pésimo:** `$sql = "SELECT * FROM usuarios WHERE email = ' " . $_POST['email'] . "'";`
(Esto permite que hackeen la base de datos en 3 segundos).
- **☑ Excelente:** Utilicen siempre los marcadores de PDO (`:email`) y el método `prepare()` seguido de `execute()`, exactamente como está en `Usuario.php`.

2. Bloqueen las puertas (Seguridad de Sesiones)

Si están programando el `JugadorControlador`, ¡no asuman que solo entrarán jugadores! Alguien podría intentar entrar copiando la URL.

- **Regla:** Dentro de la función `__construct()` de cada nuevo controlador, llamen a `Sesion::requerirLogin();` y luego verifiquen que el `$_SESSION['rol_id']` sea el correcto (tal como se hizo en `AdminControlador`).

3. Nombres que se expliquen solos (Clean Code)

Escribimos código para que lo lean otros humanos, no solo las máquinas.

- **✗** No usen variables como `$x`, `$data` o `$res`.
- **☑** Usen nombres claros: `$listaTorneos`, `$usuarioEncontrado`, `$puntajeFinal`.
- Además, ¡comenten su código! Pongan un `//` explicando *por qué* hicieron algo, no *qué* hace (eso ya se lee en el código).

4. Git: Descargar antes de Subir

El mayor problema al trabajar en equipo es borrar accidentalmente el código de un compañero.

- Antes de empezar a programar en el día, ejecuten siempre: `git pull`.
- Antes de subir sus cambios (`git push`), vuelvan a hacer `git pull` para asegurarse de que nadie subió nada mientras ustedes programaban. Si hay un conflicto, avisen por el chat del equipo.

5. Cuidado con el HTML suelto (Cierre de etiquetas)

Un solo `</div>` que se olviden de cerrar en una vista puede destruir por completo el menú lateral de la plantilla general. Sean muy ordenados con la tabulación (indentación) de su HTML para poder detectar fácilmente dónde se abre y dónde se cierra cada etiqueta.